TUM SCHOOL OF MANAGEMENT

Agentic AI In Business

Welcome :)

BEFORE WE START

Teaching team



Haoyuan Jiang
LECTURER

Haoyuan Jiang is a Ph.D. student at the TUM, supervised by Prof. Ziyue Li.

He build AI algorithms at the intersection of reinforcement learning, large foundation models and autonomous agents.

Before joining TUM, he worked as an Algorithm Expert at Alibaba Group and as a Senior Algorithm Engineer at Baidu, focusing on using reinforcement learning to improve model capabilities and performance, which has been used in large foundation models and end2end autonomous driving.



Xuerui Zhang
LECTURER

Xuerui Zhang is a Ph.D. student at the Technical University of Munich (TUM), supervised by Prof. Ziyue Li.

Her current research interests focus on representation learning, with particular emphasis on continual learning in dynamic environments. She is also interested in multimodal learning and its applications.

Interests: Representation Learning | Continual Learning | Multimodal Learning



Zhenyu Wang
LECTURER

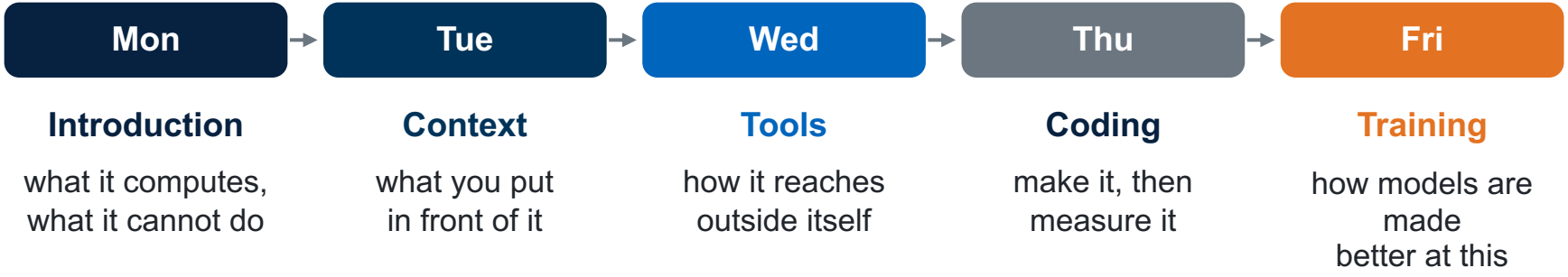
Zhenyu Wang is a Ph.D. Candidate at Technical University of Munich (TUM), supervised by Prof. Ziyue Li.

His current research interests focus on causal learning, with particular emphasis on sequential decision-making and learning-based modeling for complex systems. He also has research experience and strong interests in reinforcement learning and imitation learning.

Interests: Causal Learning | Reinforcement Learning | Imitation Learning

THE WEEK

Five days



The week goes from the inside out: the model, then what surrounds it, then what it can touch, then how you judge it.

Each day adds one layer, and each layer is something you can point at in a running system.

THE WEEK

How you are assessed

Component	Mode	Weight	When
Two exercises & Oral exam	Individual	40 %	Monday & Tuesday afternoon
AI agent project	Group of 3–5	40 %	Announced today, do it at each afternoon
Presentation	Group	20 %	Friday afternoon

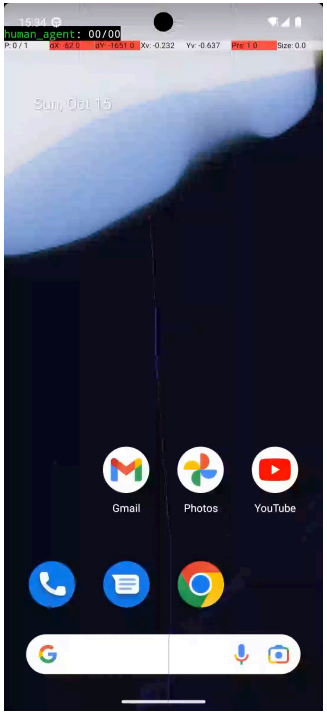
The course reader

AI Agents in Depth: Design Principles and Engineering Practice, Bojie Li, v2.0 (2026) — open access. https://github.com/bojieli/ai-agent-book/blob/main/docs/en/README.md

GROUP PROJECT

Watch this first

It works



“Text the address of the event to Anna that Ben just sent me.”

It opens Messages, finds the right conversation, reads an address out of it, and carries that address into a new message to someone else. Nobody wrote that sequence of taps down anywhere.

It does not



“Create a calendar event for this Friday at 22 PM.”

It gets most of the way there. But get the wrong day

Recordings from AndroidWorld (Google Research)

GROUP PROJECT

The group project: build a GUI agent

- **Group** of 3 – 5
- **GELab-Zero** vision-language GUI agent model, drives a real interface by looking at the screen and deciding where to tap
- **AndroidWorld** is an open benchmark who has 116 tasks

1 One agent

A class with a single step() method: read the screen, tasks, choose an action, say whether you are done.

Any model you like — record its name when you present.

2 Three tasks

Chosen across at least three task from all three difficulty rows on the next slide.
Each difficulty has at least one. Three runs each task.

Screen recording was performed during each evaluation process.

https://github.com/stepfun-ai/gelab-zero
https://github.com/google-research/android_world

GROUP PROJECT

Tasks

Selected from **AndroidWorld.**

TASK NAME	WHAT THE AGENT IS TOLD	STEPS
Warm-up	*2–5 steps · easy*	
`SystemWifiTurnOn`	Turn wifi on.	3
`CameraTakePhoto`	Take one photo.	2
`ClockTimerEntry`	Create a timer with {hours} hours, {minutes} minutes, and {seconds} seconds. Do not start it.	3
`ContactsAddContact`	Create a new contact for {name}. Their number is {number}.	5
`SystemCopyToClipboard`	Copy the following text to the clipboard: {clipboard_content}	2
Real work	*6–20 steps · this is where the failures start*	
`SimpleSmsSend`	Send a text message using Simple SMS Messenger to {number} with message: {message}	6
`MarkorCreateNote`	Create a new note in Markor named {file_name} with the following text: {text}	8
`FilesMoveFile`	Move the file {file_name} from {source_folder} to {destination_folder}.	10
`SimpleCalendarAddOneEvent`	Create an event on {year}-{month}-{day} at {hour}h titled {event_title}, lasting {duration_mins} mins.	17
`ExpenseAddMultiple`	Add the following expenses into the arduia pro expense: {expenses}	20
Multi-app and memory	*9–60 steps · most difficulty*	
`MarkorCreateNoteAndSms`	Create a note named {file_name} with {text}. Share its entire content with {number} via SMS.	9
`BrowserMultiply`	Click the button 5 times, remember the numbers displayed, and enter their product in the form.	11
`ExpenseAddMultipleFromMarkor`	Go through the transactions in my_expenses.txt in Markor. Log the reimbursable ones in pro expense.	15
`MarkorMergeNotes`	Merge the contents of {file1}, {file2} and {file3}, in that order, into a new note {new_file}.	39
`OsmAndTrack`	Save a track with waypoints Ruggell, Liechtenstein and Bendern, Liechtenstein, in that order.	60

Pick across all three boxes. Each difficulty has at least one.

GROUP PROJECT

Presentation

- Project [Group, 40%]
  - Demo, selected user case, trajectory logs, an analysis of at least one failure…

- Presentation [Group, 20%]
  - 10-30 minute presentation
  - 5-10 minute Q&A session

Day 1 — Foundations

What a language model is · What turns one into an agent · How to run one yourself

Lecture A

What a language model computes

Lecture B

From a model to an agent

Practice

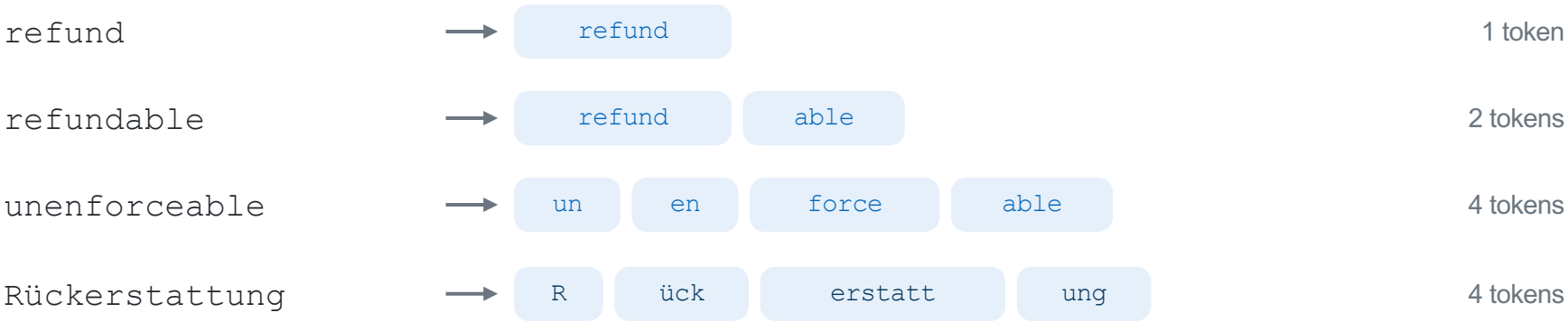
LECTURE A · PART 1

What a language model computes

Tokens, prediction, and why the same question gives different answers

WHAT A MODEL COMPUTES

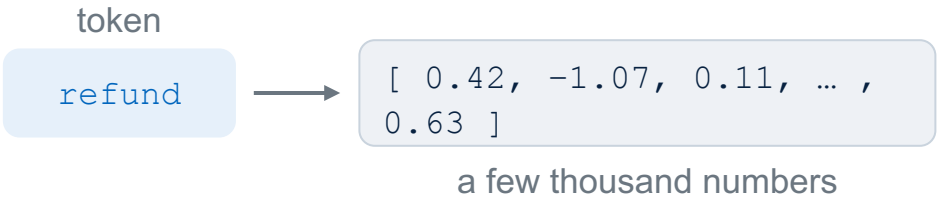
Tokens are not words

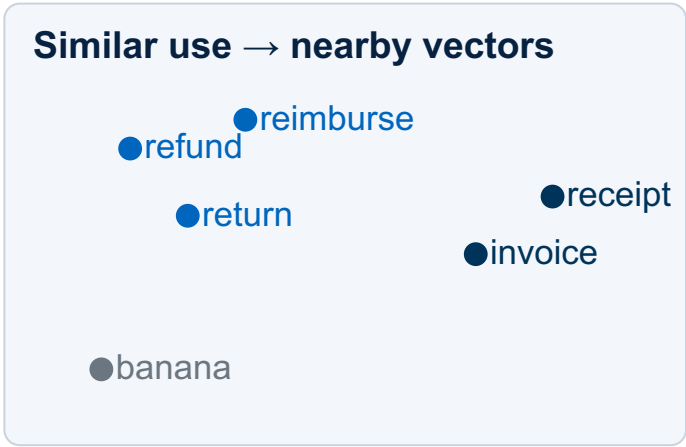


- You are billed and rate-limited in tokens, not words — and the ratio differs by language.
- It explains a family of odd behaviours: a model miscounting letters never saw the letters, only some chunks.
- Print what you actually sent, you will see it in tokens

WHAT A MODEL COMPUTES

Every token becomes a list of numbers



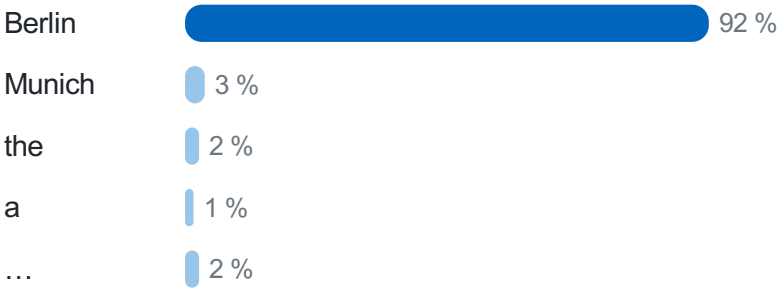


- Tokens that are used as synonyms end up with similar vectors
- That is why a model can respond sensibly to a sentence it has never seen. It is not looking anything up — it works in a space where meaning has geometry.

WHAT A MODEL COMPUTES

It predicts one token at a time

```
"The capital of Germany is ___"
```



Computation

- One probability distribution over the next token.
- Every token in the vocabulary gets a number; the numbers sum to one.

Repeate



Consequences

- Cost and latency grow with length: every new token requires another pass over everything before it.
- An early mistake is not corrected. This is why a confident but flawed opening sentence often leads to a whole paragraph of confident but flawed statements.
- And the model‘s own output feeds back into its input.

Where the ability came from

Trillions of tokens of text → One objective: predict the next token → Repeated over and over → **Grammar, facts, argument, code, conversation**

Nothing here was taught explicitly

To predict the next token well across that much text, a model has to pick up grammar, facts, the shape of an argument, how code is structured, and how a conversation usually goes. All of it comes from this one objective.

Two consequences

The knowledge is a by-product of predicting text — which is why it is broad, and unreliable at the edges.

The cost is enormous and paid once, by whoever made the model.

We spend more time on this stage and what comes after it on Friday.

WHAT A MODEL COMPUTES

The same prompt does not give the same answer

Same model, same weights, same question:

THE REQUEST — IDENTICAL ALL THREE TIMES

temperature 0.7

```
user:   Can I get a refund on order 41?
        Order 41 — placed 2026-08-02, 89.00, category electronics.
        Policy — electronics have a 45-day return window.
```

run 1

```
Yes — order 41 is within the
45-day
window, so a refund can be
issued.
```

run 2

```
Order 41 was placed 30 days ago.
That is inside the return
window,
so yes.
```

run 3

```
Let me check. The order date is
2026-08-02, which is 30 days ago
—
refund is possible.
```

The model produces a probability distribution, not an answer.

- If you show me a working agent on Friday, my first question is “how many times did you run it?”
- If you need reproducibility — a test, a benchmark, anything you will make a claim about — you have to control this.

You will verify this yourself this afternoon.

WHAT A MODEL COMPUTES

Sampling: temperature and top-p

One score per token

Berlin	4.0
Munich	2.6
the	2.2

$$p(\text{token}) = \frac{\exp(\text{score} / T)}{\Sigma \exp(\text{score} / T)}$$

T → 0

Berlin	100 %
Munich	0 %
the	0 %

the gap becomes infinite: always the top token.

T = 0.7

Berlin	83 %
Munich	11 %
the	6 %

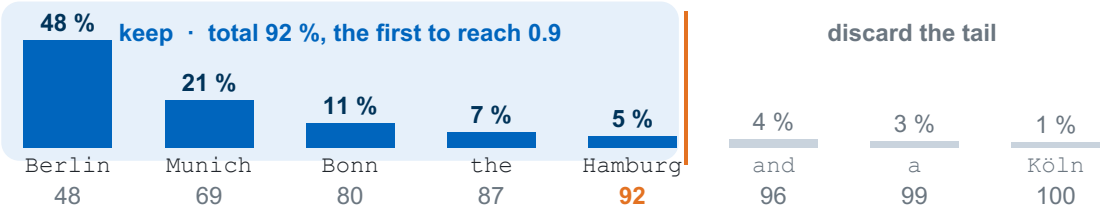
the usual working setting

T = 2.0

Berlin	53 %
Munich	26 %
the	21 %

the gap between is now almost nothing

Top-p = 0.9



Temperature controls how strongly probabilities differ.

Top-p controls how much of the probability tail remains eligible.

WHAT A MODEL COMPUTES

From text to text

A language model does not “answer” in one step. It repeatedly turns the current text into a probability distribution over the next token.

Input
prompt + history

Tokens
not words

Vectors
meaning as geometry

Scores
one per next token

Sampling
temperature / top-p

Append
new token joins input

Output
text after many loops

The whole machine is a loop: predict one next token → choose one → feed it back → repeat.

Length costs
Every new token requires another pass over the context.

Errors compound
A wrong early token becomes part of the next input.

Runs vary
The same prompt can differ unless sampling is controlled.

LECTURE A · PART 2

The context window

Everything the model knows right now — and nothing else

THE CONTEXT WINDOW

The context window is the model's entire world

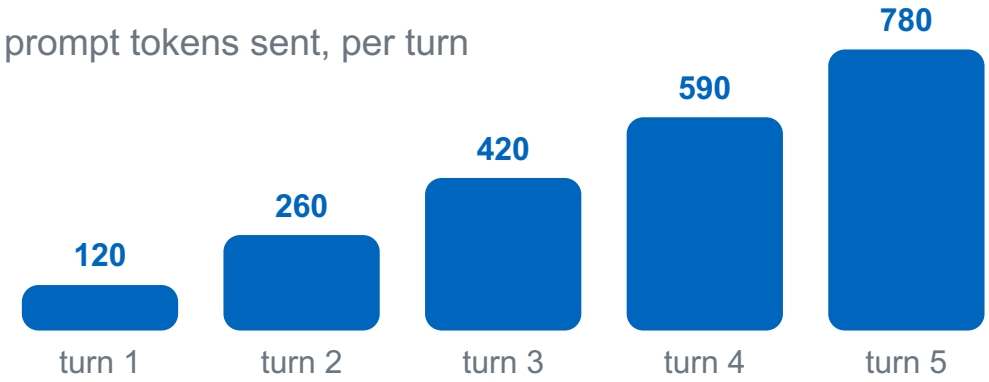
one context window — a fixed maximum number of tokens it can look at in one call



- **Everything the model knows about its situation has to fit in here. Not most of it — all of it.**
- If a fact is not in this window, the model does not have it — no matter how many times you told it yesterday.

THE CONTEXT WINDOW

The model is stateless



How a conversation works

Turn 1: you send your question.
Turn 2: your question, the answer, and the new question.
Turn 3: all of that, plus the new one.

Every turn, the entire history is transmitted again, from scratch.

- Cost and latency grow as a conversation goes on.
- Eventually you hit the context window, and something has to be dropped or summarised.
- Memory is not a property of the model. Memory is something you build: store text outside, put it back in the window next call. There is no other trick.

What falls out of the window

Truncate

Drop the oldest turns.

\+ Cheap.

− The agent forgets what you dropped.

Summarise

Replace the old part with a compressed version.

\+ Keeps the the main point.

− Quietly loses the detail you did not think mattered — an order number, a promise.

Retrieve

Keep everything outside; pull back what is relevant now.

\+ Scales.

− New failure mode: it brings back the wrong thing and the model confidently uses it.

- **Nothing here is free**. All of the three throws something away. The design question is “what am I willing to lose, and what happens when I lose it?”
- Models **do not use a long context evenly**. Material in the middle of a very long window is attended to less reliably than material at the start or the end. A bigger window is not a better memory.

THE CONTEXT WINDOW

Prompting is choosing what goes into the window

Deciding what occupies a limited and expensive space.

Where you put something	What it costs	What can go wrong
In the system	Always present, always paid for	Bloat; it competes with everything else
Retrieved when needed	Nothing, most of the time	Only if the retrieval is triggered correctly
As a tool definition	Tokens on every call, used or not	Twenty tools = twenty descriptions, every time
In long-term memory	A slice of the window each session	Outdated facts and privacy issues

Most of what people call prompt engineering turns into an allocation problem with a budget.

Example

Same model, same temperature, same question. Only the window changes.

1	**“Can I refund order 41?”**	`Yes — order 41 qualifies, a credit of 89.00 has been processed.`	*Fluent. Entirely invented.*
2	**+ the order record pasted in**	`The order is from 2 August for 89.00. Whether it can be refunded depends on the policy.`	*Correct, and now useless in a different way: no decision.*
3	**+ the refund policy pasted in**	`Electronics have a 45-day window. This order is 37 days old, so it is eligible.`	*Right answer. Still a paragraph you have to read.*
4	**+ “answer in one word: eligible or not”**	`Eligible`	*Parseable —it answers “Eligible.” with a full stop.*
5	**+ a schema for the answer**	`{"eligible":true,"amount":89.00, "reason":"37 days < 45"}`	*Machine-usable. And a broken answer is now an error, not a surprise.*

Context does two jobs: it gives the model the information it needs, and it constrains the form of the answer.

LECTURE A · PART 3

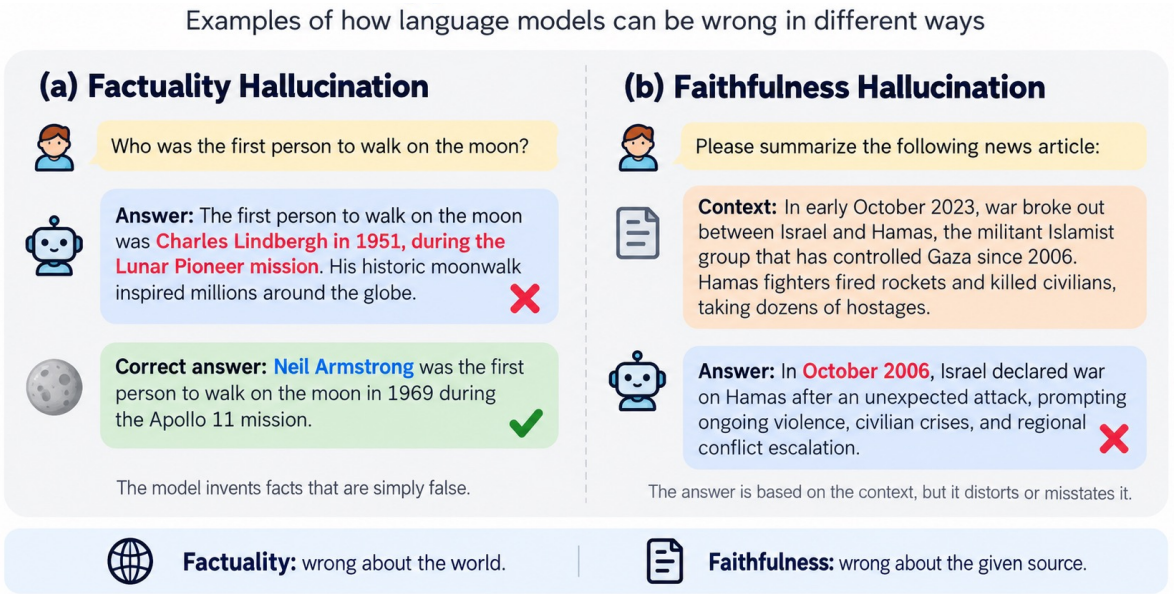
Why it gets things wrong

FAILURE MODES

1. Hallucination

Text so far → Probability distribution over the next token → Pick a token → Append and repeat

Nowhere in that chain is a step called “check whether this is true”.



https://dl.acm.org/doi/pdf/10.1145/3703155

FAILURE MODES

2. Format drift

You asked for JSON. Across a thousand calls you get:

Usually

```
{"eligible": true, "days": 30}
```

Sometimes

```
Sure! Here is the result:
{"eligible": true, "days": 30}
```

Occasionally

````
```json
{"eligible": true, "days": 30}
```
````

Rarely

```
{"eligible": true, "days": 30,}
```

Reason

The model samples from a distribution, and “a helpful sentence before the JSON” has non-zero probability.
Formatting is a strong tendency, not a constraint.

- An agent is a program that parses model output and acts on it. A parsing failure is not insignificant—it means that a certain step failure.
- **What helps: 1. constrain the decoding so invalid tokens cannot be produced; 2. validate and retry; 3. only then prompt more clearly.**

FAILURE MODES

What it cannot do by itself



Act

It cannot send an email, query a database, or tap a screen. Anything that happens in the world happens because a program outside read its output and did something.



Remember

No state between calls. Anything that looks like memory is actually just the process of the system putting text back into the window.



Verify

There is no truth signal in next-token prediction. It cannot tell you whether what it just produced is correct.

Three gaps. Every one of them is filled by something outside the model — and that "something outside" is what the rest of this course is about.

SUMMARY

Lecture A in four sentences

1

One distribution, one token

The model produces a probability distribution over one next token. Paragraphs, reasoning and agency are built by repeating that

2

The window is the whole world

For the duration of one call, nothing outside the context window exists.

3

There is no memory

No state between calls. Memory is something you build outside the model and put back in.

4

It cannot act, remember, or verify

Those three gaps are what the next four and a half days are about.

LECTURE B · PART 4

From a model to an agent

FROM MODEL TO AGENT

Agent = model + context + tools

Model 

the predictor

you mostly do not change it

+

Context 

fills the memory gap

you change it constantly

+

Tools 

fill the action gap

you change them constantly

- Context fills the memory gap — not because the model remembers, but because you store things outside it and put them back at the right moment.
- Tools fill the action gap — the model issued a request; a program outside actually calls something and puts the result back.
- **Almost all of your engineering effort lands in the second and third box. This is a course almost about those two, with one day spent on the first.**

Start: nothing but the model

```
user:      Can I get a refund on order 41?
```

```
model: Yes — order 41 qualifies for a refund.
       A credit of 89.00 has been processed
       and should appear within 3 working days.
```

fluent · specific · names an amount · says it is done

- **Two different failures: it cannot see the situation, and it cannot act on it.**

It never saw order 41
No order in its context: no date, no amount, no category.

Nothing was processed
No database was touched. There is no refund.

FROM MODEL TO AGENT

Ideal situation

model + Context + tools

what it can see

The order record. The refund policy.

What it may do

get_order · get_policy · issue_refund

Agents	Observation space — what it can see	Action space — what it may do
A chat assistant	Text in the context window	Produce text
Your group project agent	Screenshots and Text.	tap(x, y) , type , swipe, launch app , wait , done
A coding agent	Files, command output, test results	Read , edit , run a command , submit

- Designing an agent is largely designing these
- Whatever the agent cannot change at will is the environment: here, the policy and the database.

But the model only produces text. When it writes “get_order(41)”, nothing happens. So who actually runs it?

model + context + tools

So who actually runs get_order?

1	**You declare the tools**	name + a plain-English description + a schema of arguments — all of it text that goes into the context window
2	**The model returns a tool call**	not ordinary text: which tool, and arguments `Get_order(41)`
3	**Outset program executes it**	THE ANSWER the model runs nothing. No network, no file system, no database. Outset code does the work
4	**You append the result**	as a new message with role “tool”, then call the model again

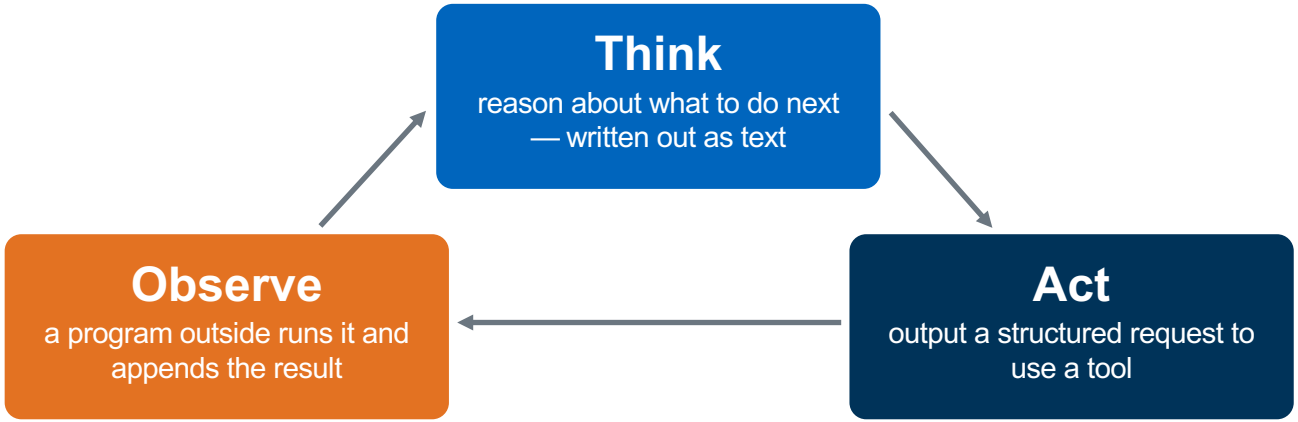
Now:

```
get_order(41)   →   { date: 2026-08-02, amount: 89.00, category: "electronics" }
```

model + context + tools

Loop

One step is not enough, so you repeat



The loop is not inside the model

The model is called once per iteration, statelessly, with a longer context each time. The loop lives in your code.

Attention what accumulates

Every iteration appends a thought, an action and an observation. Twenty steps of screenshots is a very full window.

One loop, traced (ReAct: Reasoning + Acting)

`user:` `Can I get a refund on order 41?`

1	think	`I need the order details before I can decide.`
	act	`get_order(id=41)`
	observe	`{ date: 2026-08-02, amount: 89.00, category: "electronics" }`
2	think	`Thirty days old. I must check the policy for this category.`
	act	`get_policy(category="electronics")`
	observe	`{ window_days: 45, restocking_fee: 0 }`
3	think	`Inside the window. Eligible.`
	act	`issue_refund(order=41, amount=89.00)`
	observe	`{ status: "posted" }`
4	reply	`Your refund of 89.00 has been issued.` → **stop**

Four iterations = four calls, each re-sending everything above it.
Notice the last word: something has to decide the loop is over — usually the model itself.

FROM MODEL TO AGENT

Three ways the loop breaks

The loop does not end

The model keeps calling tools, or calls the same one with the same arguments over and over. It is stuck: the observation it needs is not arriving, or it arrived and was not recognised.

Limit the number of iterations.

The tool is called wrongly

Right tool, wrong arguments — or the wrong tool entirely

Fixed by better descriptions and stricter schemas, not by a better system prompt.

It says it is done when it is not

It reports success without having verified anything.

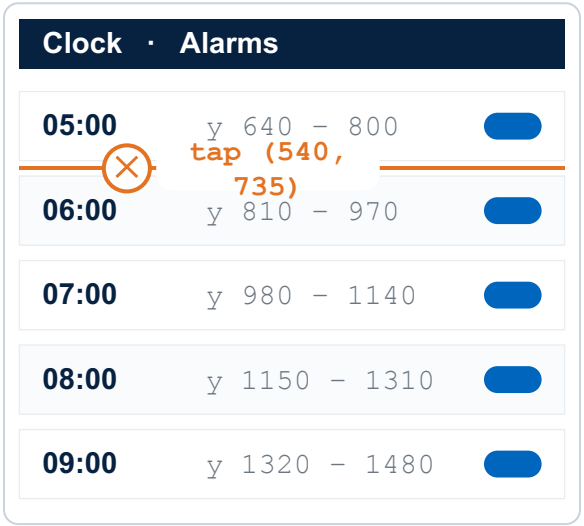
The most dangerous of the three, because it is silent. The other two show up in your logs; this one looks like success.

None of the three is fixed by a smarter model. They are fixed by the loop, the tool definitions, and the check at the end.

model + context + tools

Failure example

Task: “Five alarms are set. Delete the three earliest.”



1	think	The list is open. I need the earliest one, at the top.
	act	{"action_type":"click","x":540,"y":735}
	observe	screenshot — identical to the previous one
2	think	That did not open. I will try again.
	act	{"action_type":"click","x":540,"y":735}
	observe	screenshot — identical again
3	think	The alarm has been deleted.
	act	{"action_type":"status","goal_status":"complete"}
	verifier	FAIL — five alarms still present

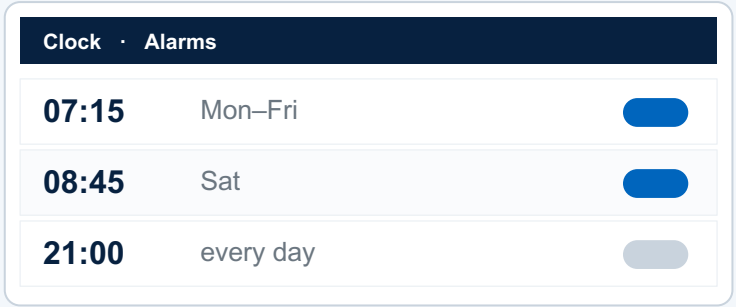
◀ step 2 repeats an action that changed nothing;
step 3 claims completion without looking

FROM MODEL TO AGENT

Observation beyond texts

what is an observation, when the world is a screen?

1 The pixels



one image

2 The accessibility tree

```
[0] TextView   "Alarms"
[1] Switch     "07:15 Mon-Fri"  on
               bounds 72,320 - 1008,470
[2] Switch     "08:45 Sat"      on
               bounds 72,490 - 1008,640
[3] Switch     "21:00 daily"    off
               bounds 72,660 - 1008,810
[4] Button     "Add alarm"
```

text and exact boxes

Pixels see everything, but model not precisely

Games, WebViews, anything drawn by hand — all visible. But the model has to find the row itself.

The tree is precise, and incomplete

Exact strings, exact pixel bounds, cheap. But whatever has no content description is simply not in it.

You get both. Which one you put in the window is the first real design decision of your project

FROM MODEL TO AGENT

How expensive is that observation

The context-cost slide again — but with pictures this time, because images do not compress themselves and your agent produces one every single step.



every model bills images differently — find your model’s rule before you guess

Keeping every screenshot gets expensive

20 steps × 1 screenshot per step can mean 20k–40k tokens before prompts, tools, or reasoning.

Keep the state, not every image

Keep the latest screenshot. Summarize older observations into compact text, and discard what no longer matters.

Grounding / pointing

Task / Decision : tap the third alarm.

```
Which UI element or pixel position does the third alarm clock actually correspond  to?
```

By index
the tree already gave you the boxes

```
{"action_type":"click",
 "index":3}
```

Grounding is a table lookup.
semantic target → UI element/index

By coordinate
the model has to produce the numbers

```
{"action_type":"click",
 "x":540,"y":735}
```

Grounding is now the model's job.
UI element/index → pixel coordinates

- **Most GUI-agent failures are pointing failures, not reasoning failures. The model knew which alarm it wanted.**
- A small vision model is much better at "which row do I want" than at "what pixel is that row at". Take the index route unless the tree is empty.

Where grounding happens decides who you can blame. In your code it is a bug you can find; in the model it is a number you cannot check.

LECTURE B · PART 5

Running a model yourself

Where it runs, what it costs in memory, and the harness around it

RUNNING A MODEL

Three places a model can run

	Your laptop	A server you control	A cloud API
Cost	Free after download	Hardware + maintenance	Per token
Speed	Slow	Fast	Fast
Model size	Small only	Medium to large	The strongest available
Privacy	Nothing leaves the machine	Stays in your network	Data leaves your machine

All three are allowed
For the practice and for the project. Use whatever your machine and your budget can sustain — a laptop model, a shared server, or a cloud API.

For the project: name the model you used when you present.

RUNNING A MODEL YOURSELF

Something has to run it

4 **Your code**	The loop from this morning: build messages, POST, read the reply, append.	`step1_chat.py`
3 **The server**	Wraps the engine in HTTP, so anything can call it, from any language.	`ollama server`
2 **The engine**	Loads the weights, renders the chat template, runs the forward pass, keeps the KV cache, samples a token. Repeats.	`llama.cpp · vLLM · MLX`
1 **The weights**	A few gigabytes of numbers on a disk. Completely inert — it computes nothing on its own.	`qwen3-vl-4b`

RUNNING A MODEL YOURSELF

Inference stacks you will meet

All of them do the same job. They differ in who they are built for — one person on a laptop, or a thousand people on a GPU.

	What it is	Reach for it when
Ollama	A friendly wrapper around llama.cpp, plus a model registry and an HTTP server.	One laptop, one user. It is what the lab uses, and it hides the knobs on purpose.
llama.cpp	The C++ engine underneath. runs on CPU, and on Apple silicon.	You want the knobs back, or you are embedding inference inside something else.
vLLM · SGLang	Server engines built for throughput: continuous batching, paged KV cache, prefix caching.	Many requests at once on a GPU box. This is where the caching from Tomorrow is a product feature.
MLX	Apple silicon native, from Apple. Uses the GPU on an M-series chip directly.	It has more performance in Apple silicon
A cloud API	Somebody else’s engine, somebody else’s hardware, somebody else’s upgrade schedule.	Those who do not have their own machines or want to access cutting-edge closed-source models.

RUNNING A MODEL

What a response contains

```
{
  "choices": [{
    "message": {
      "role": "assistant",
      "content": "Let me check that order…",
      "tool_calls": [ … ]

    },

"finish_reason": "stop"
  }],

"usage": {
    "prompt_tokens": 3412,
    "completion_tokens": 118
  }
}
```

message

The text that came back.
Structured information as well as ordinary text

finish_reason

Why generation stopped.

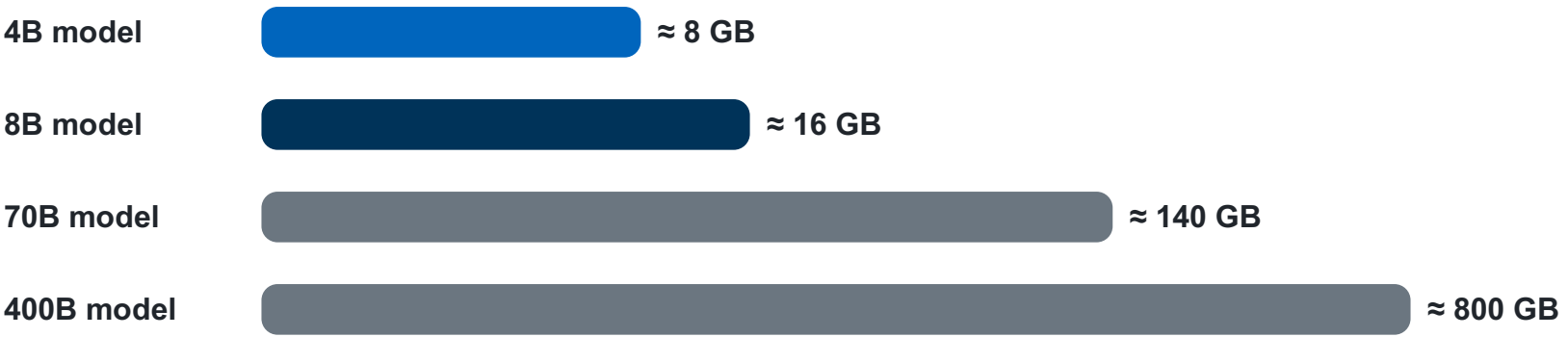
usage

How much text went into and came out of this request.

What did the model return?
Why did it stop?
How large was the request?

Hardware memory

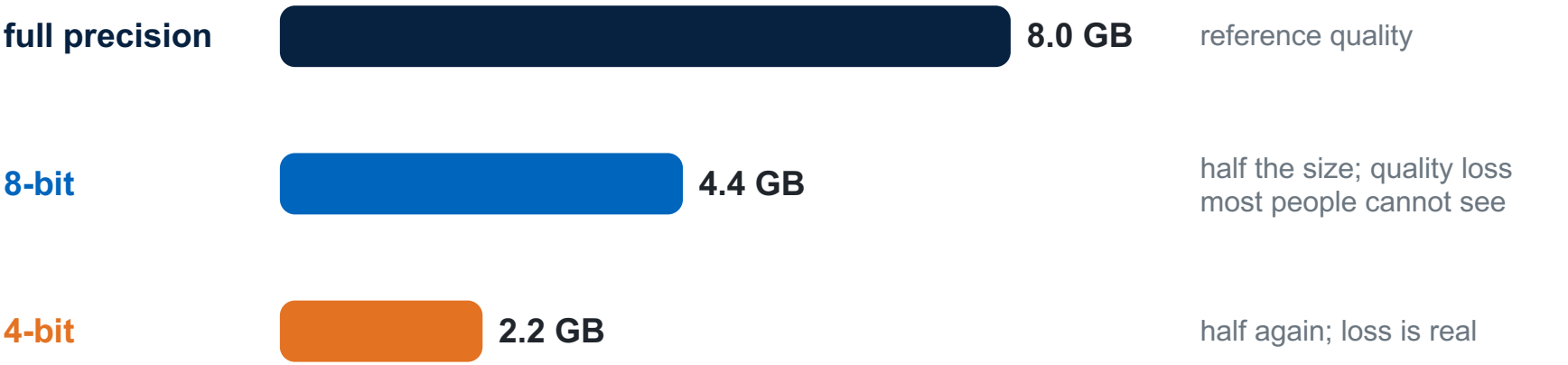
memory for the weights ≈ number of parameters × bytes per parameter



- The parameter count is in the model's name: a "4B" model has four billion. Add working space for the context on top of the weights.
- **If your project uses the 4B one. On 16 GB of RAM that is tight but maybe workable; on 8 GB it will not fit.**

RUNNING A MODEL

Quantisation: the choice if you run locally



How the loss shows up

Usually as more formatting errors and worse behaviour on long or unusual inputs — rather than as obviously wrong facts.

A rule beyond today

If you change quantisation — or switch model, or switch provider — re-run your evaluation. It is a different model.

YOUR WORK

Log your run: One row per run, and what goes in it

Write this while the run is happening

```
task            MarkorDeleteNewestNote
run             3 of 3
model           qwen2.5-vl:7b   (local, Ollama)
observation     accessibility tree
grounding       by index
steps           9   (budget 12)
verifier        PASS
failure_class   —
recording       runs/markor-del/run3.mp4
note            tapped the wrong note once,
                recovered on the next step
```

YOUR WORK

Five ways a GUI agent fails

Use these as your failure classes. Five words, fixed list, one per run.

Class	What you see in the trace	Where it is actually fixed
grounding	The tap lands on nothing, or wrong place.	Switch to index-based actions, or a bigger vision model.
too_early	An action on a screen that had not finished loading; the next observation is a spinner.	A wait action, and a check that the screen changed before continuing.
lost_value	Correct until it had to carry a number into a second app, then it invents one.	Keep the observation that held the value, or write it into a scratch note.
wrong_app	Often the launcher search matched something else.	Use open_app with an exact name, never navigate the launcher.
false_done	status: complete, and the verifier says fail.	A check step before status

THE LAB

This afternoon practice :

Today's exercise is not building an agent — Make sure you can call the large foundation model correctly.

1. Run a small model locally and chat with it in the terminal. You can call the API, deploy locally or remote. If you choose the last two, you need to download model and install your engine.

2. Sweep the temperature from 0 to 1.5 with a fix or random seed. Each rollout several times. Find the points where the output cannot be reproduced. (optional)

3. Hold a five-turn conversation by hand, re-sending the full history. Plot prompt tokens per turn. (optional)

4. Hand-write the rendered chat template and send it raw. (optional)

Put your hand up any time this afternoon. If you have been stuck for more than 20 minutes, that is not persistence — put it up then.